HTTP Headers Analysis Report

1. Introduction

This report is about analysing HTTP headers. For this assignment, I used wireshark, a network packet sniffer, as my application for analysis.

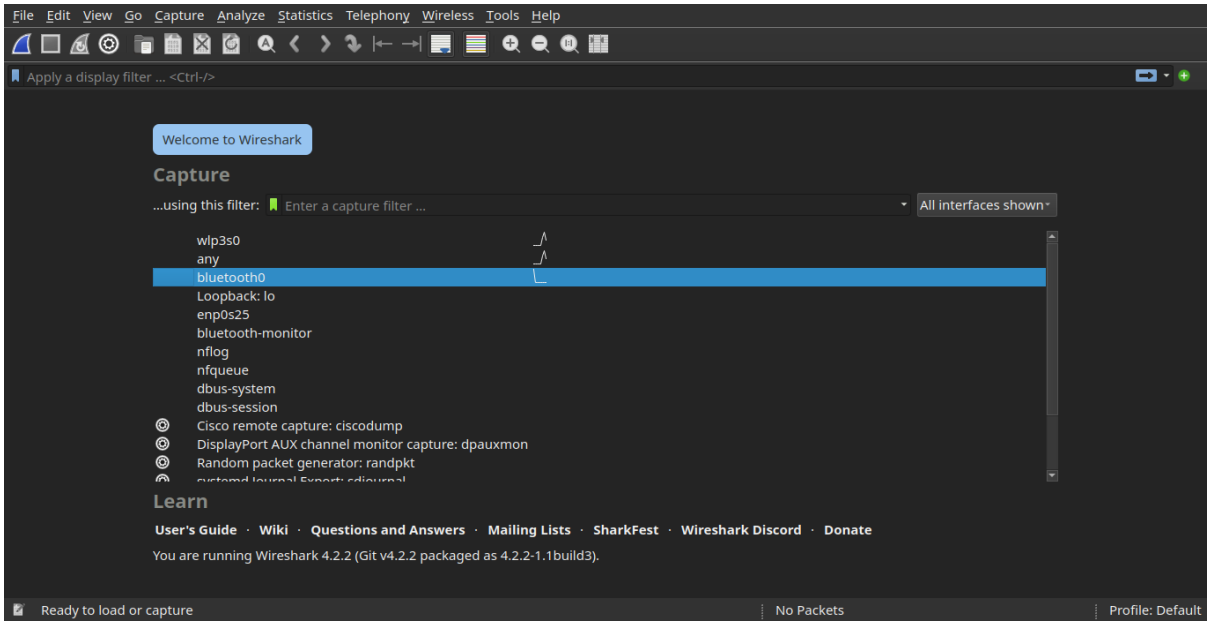
```
gabriel-kimani@gabriel-kimani-ThinkPad-X240:~$ sudo apt-get install wireshark
[sudo] password for gabriel-kimani:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wireshark is already the newest version (4.2.2-1.1build3).
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
```

2. Methodology

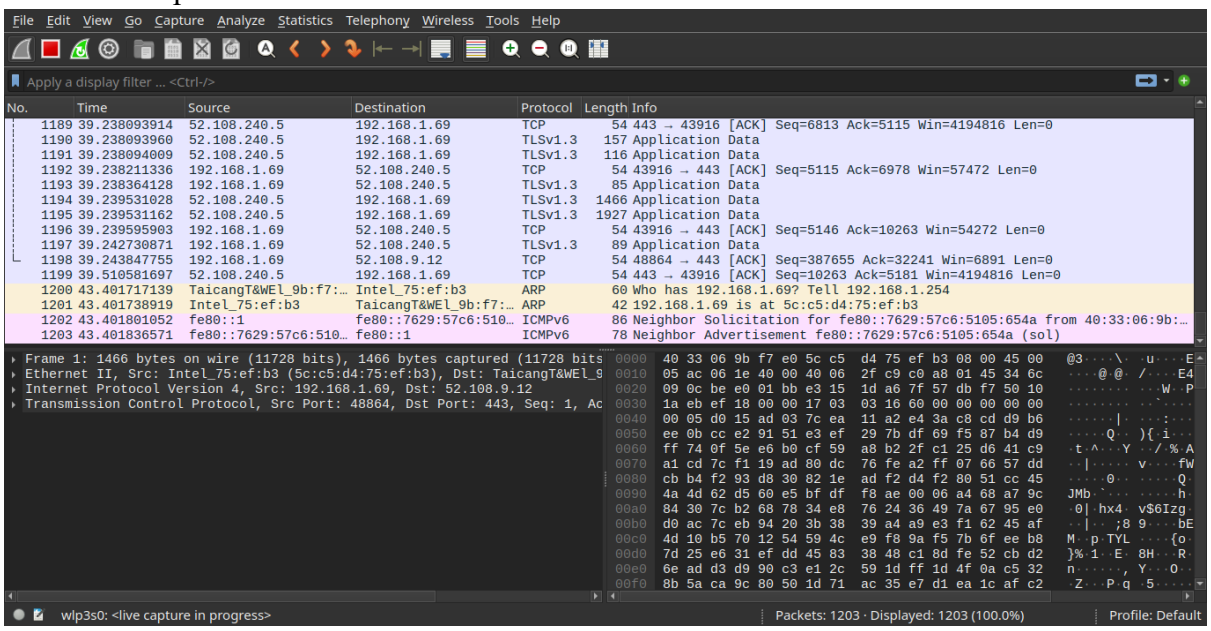
- My set up is as follows:
    - Operating System: Linux (specifically Ubuntu)
    - Wireshark version: Wireshark version 4.2.2-1build3
    - Browser used: Mozilla Firefox
- To use the software, I had to run it with higher privileges using the sudo command as follows:

```
gabriel-kimani@gabriel-kimani-ThinkPad-X240:~$ sudo wireshark
 ** (wireshark:9749) 23:12:47.500751 [GUI WARNING] -- QStandardPaths: XDG_RUNTIM
E_DIR not set, defaulting to '/tmp/runtime-root'
```

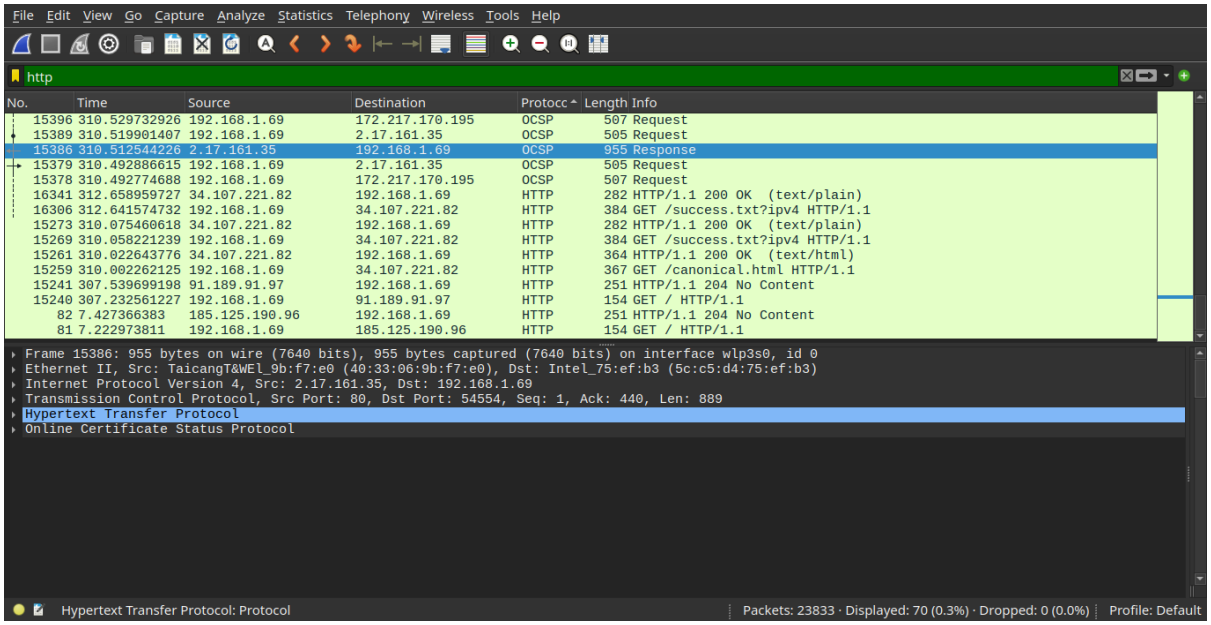
- This takes me to the main window where I have a list of various interfaces.



- I opted for the wlp3so (but any is better for post) and chose it by double tapping it, for wireless connections such as Wi-Fi. This took me to where the analysing actually takes place as follows.



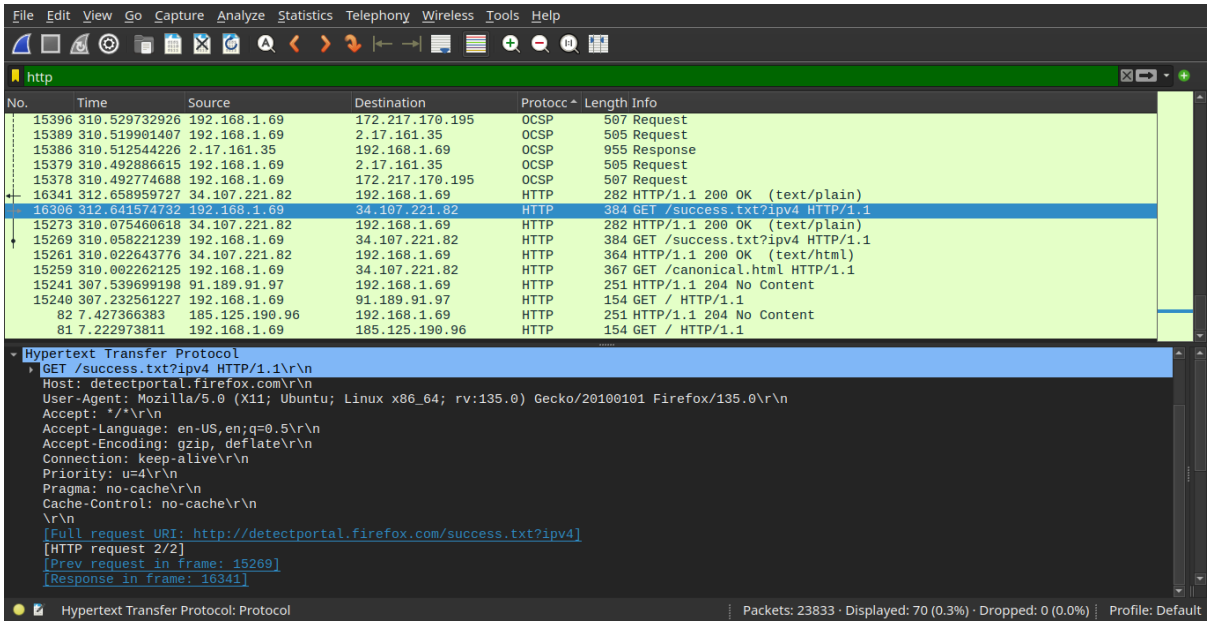
- For easier analysis, I applied a filter(https) from the filter bar, and stopped further capturing of packets to make analysis of HTTP requests easier as follows.



3. GET Request Analysis

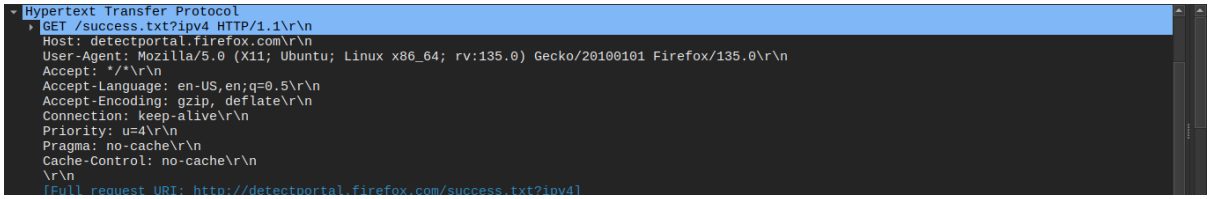
3.1 Capture Process

Here is a screenshot of the main window showing the selected GET request.



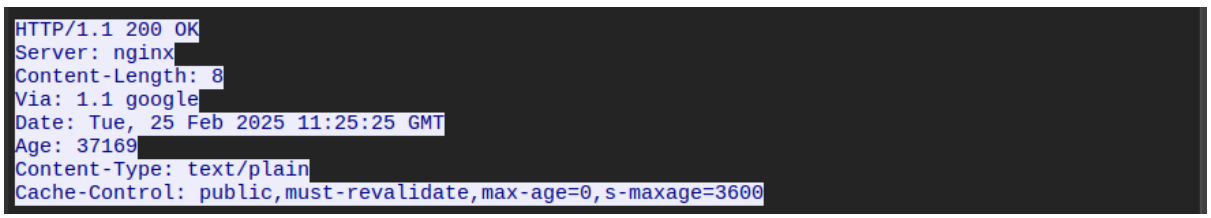
3.2 Request Headers

I right clicked on the GET request from the display pane, and followed the HTTP stream to get the following:



- Analysis of important headers:
  - GET /success.txt?ipv4 HTTP/1.1: This line indicates a GET request, which is used to retrieve data from the server. The requested resource is /success.txt with a query parameter ipv4. The HTTP/1.1 specifies the HTTP protocol version, enabling persistent connections.
  - Host: detectportal.firefox.com: This header specifies the domain name of the server being requested. It's crucial for virtual hosting, where multiple websites share the same IP address.
  - User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:135.0) Gecko/20100101 Firefox/135.0: This header provides information about the client's browser and operating system. Here, it identifies the client as Firefox 135.0 running on Ubuntu Linux.
  - Accept: */*: This header indicates that the client accepts any content type. While broad, it suggests the client isn't expecting a specific format.
  - Accept-Language: en-US, en; q=0.5: This header specifies the client's preferred languages, with US English being the most preferred and generic English being the secondary preference.
  - Accept-Encoding: gzip, deflate: This header indicates that the client can accept compressed content using gzip or deflate encoding, which helps reduce data transfer size.
  - Connection: keep-alive: This header suggests that the client wants to maintain a persistent connection with the server, reducing the overhead of establishing new connections for subsequent requests.
  - Priority: u=4: This header likely indicates the priority of the request. The u=4 value suggests a specific priority level, which could be used for resource allocation on the server-side.
  - Pragma: no-cache and Cache-Control: no-cache: These headers instruct the server and any intermediary proxies not to cache the response. This ensures that the client always receives the latest version of the resource.
  -

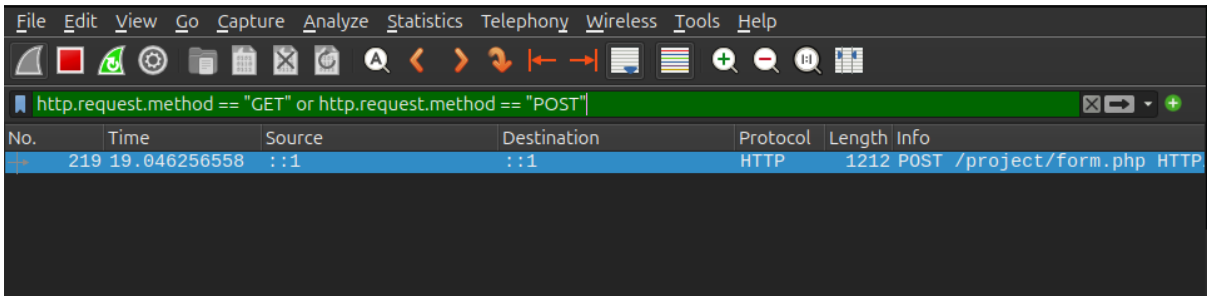
3.3 Response Headers



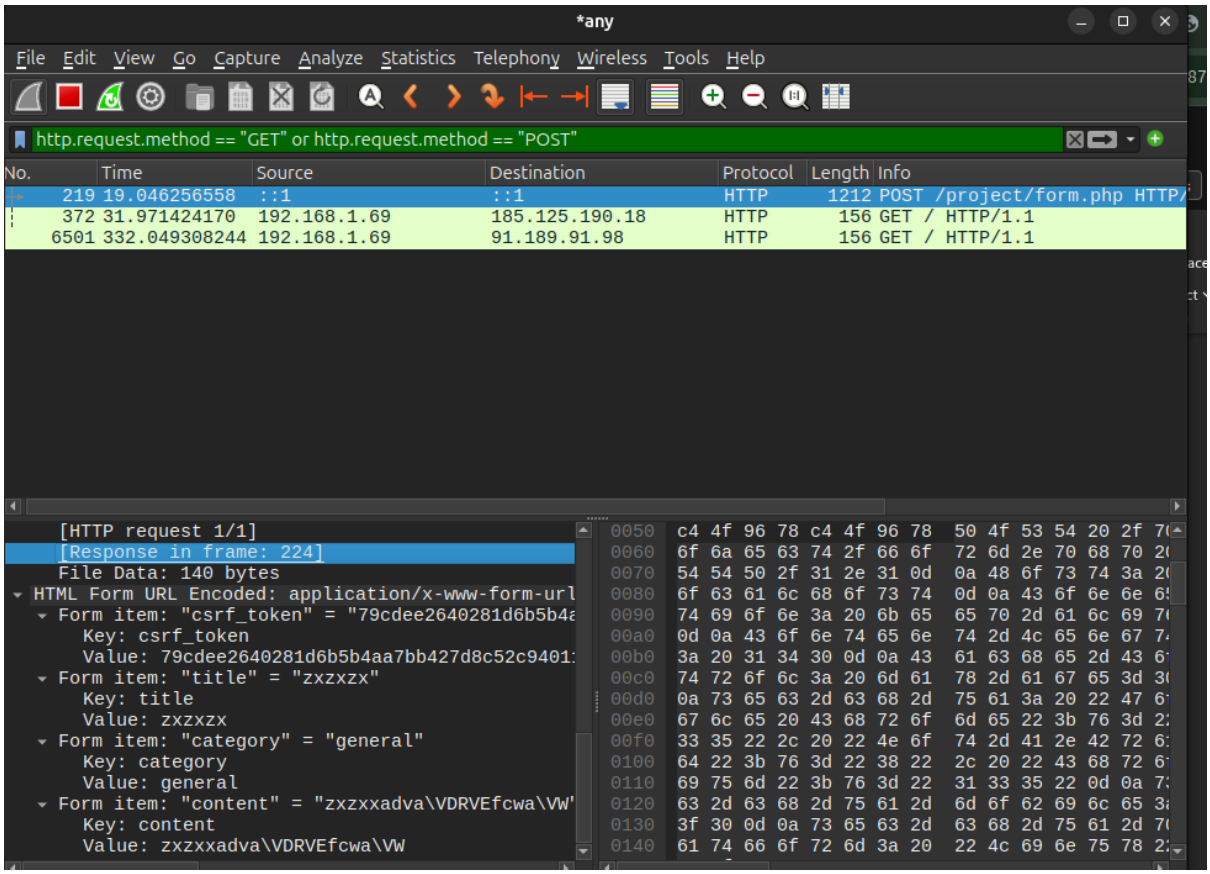
- Analysis of headers was as follows:
    - HTTP/1.1 200 OK: This status line indicates a successful HTTP request. The 200 OK status code means the server processed the request successfully and is returning the requested resource.
    - Server: nginx: This header identifies the web server software being used, which is Nginx in this case. Nginx is known for its high performance and efficiency.
    - Content-Length: 8: This header specifies the size of the response body in bytes, which is 8 bytes in this case. This is useful for the client to know how much data to expect.
    - Via: 1.1 google: This header indicates that the response passed through a proxy server, likely a Google proxy. This is common for requests that go through content delivery networks (CDNs) or load balancers.
    - Date: Tue, 25 Feb 2025 11:25:25 GMT: This header provides the date and time when the response was generated by the server, in Greenwich Mean Time (GMT).
    - Age: 37169: This header indicates how long the response has been cached by a proxy server (in seconds). In this case, the response has been cached for 37169 seconds, which is a significant amount of time.
    - Content-Type: text/plain: This header specifies the MIME type of the response body, which is plain text in this case. This tells the client how to interpret the data.
    - Cache-Control: public, must-revalidate, max-age=0, s-maxage=3600: This header provides instructions for caching the response.
    - public: Indicates that the response can be cached by any cache.
    - must-revalidate: Instructs caches to revalidate the cached resource with the origin server before using it.
    - max-age=0: Specifies that the cached response should be considered immediately stale, requiring revalidation.
    - s-maxage=3600: Specifies a maximum age of 3600 seconds (1 hour) for shared caches (like proxies). This overrides max-age for shared caches.

4. POST Request Analysis

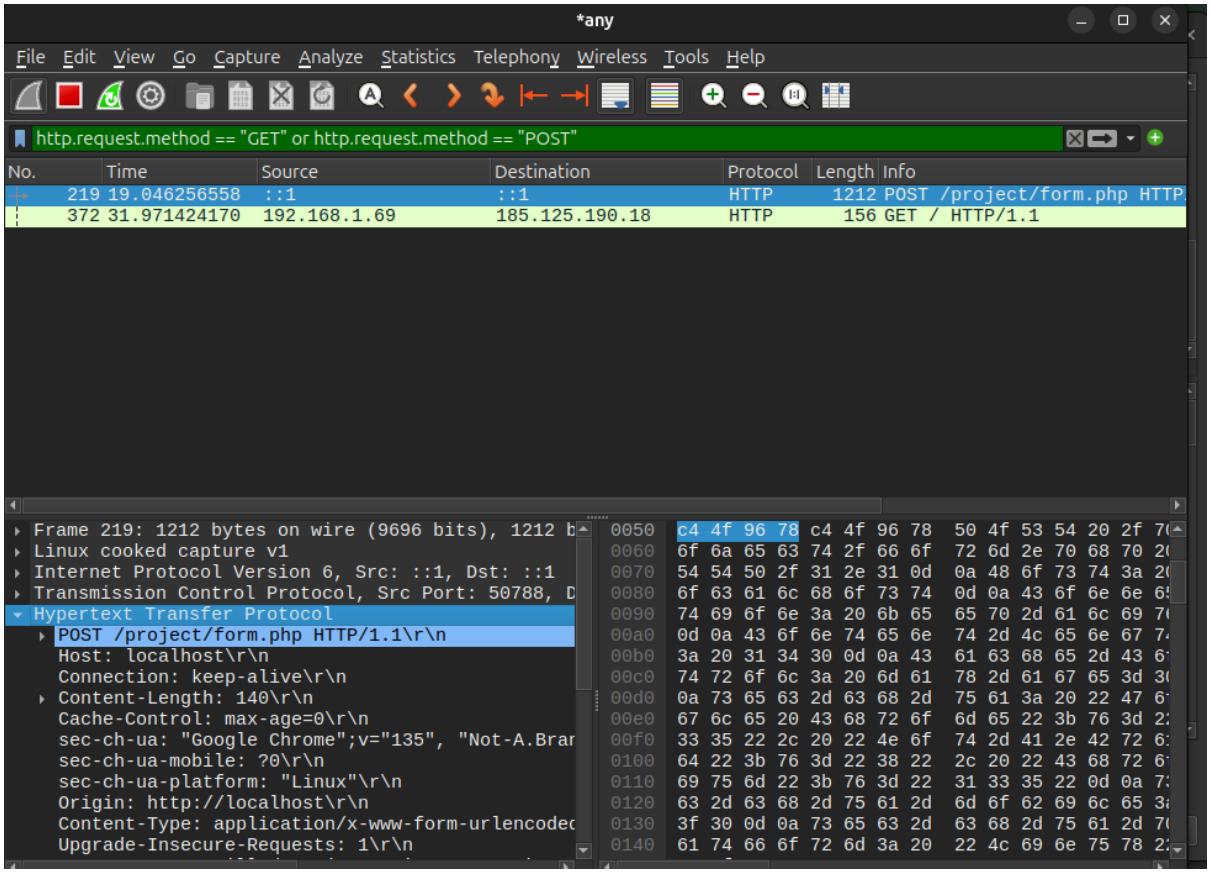
4.1 Capture Process



- The form I used was the form.php file from my project, that used the post request to show this information. It posted information about the name, category and description of a user's submission form as shown below.



4.2 Request Headers



- **Request Line (POST ... HTTP/1.1)**: Specifies the method (POST), target path (/project/form.php), and HTTP version (1.1).
- **Host: localhost**: Identifies the intended server domain (localhost).
- **Connection: keep-alive**: Asks the server to keep the TCP connection open.
- **Content-Length: 140**: Indicates the size of the data being sent (140 bytes).
- **Cache-Control: max-age=0**: Tells caches to treat the resource as immediately stale.
- **sec-ch-ua: ...**: Client Hint detailing the browser brand and version (Chrome 135).
- **sec-ch-ua-mobile: ?0**: Client Hint indicating it's not a mobile device.
- **sec-ch-ua-platform: "Linux"**: Client Hint specifying the operating system (Linux).
- **Origin: http://localhost**: Specifies the origin (scheme, host, port) that initiated the request.
- **Content-Type: application/x-www-form-urlencoded**: Defines the format of the data being sent (standard web form encoding).
- **Upgrade-Insecure-Requests: 1**: Signals client preference for an HTTPS connection.
- **Accept: ...**: Lists the content types (like HTML, XML, images) the client can understand.
- **Sec-Fetch-Site: same-origin**: Indicates the request is for the same site that initiated it.
- **Sec-Fetch-Mode: navigate**: Specifies the request mode (navigation via form submit/link click).
- **Sec-Fetch-User: ?1**: Shows the request was triggered by user interaction.

- **Sec-Fetch-Dest: document**: Indicates the response is expected to be a browser document.
- **Referer: http://localhost/project/**: The URL of the page that initiated this request.
- **Accept-Encoding: gzip, deflate, br, zstd**: Lists the compression methods the client supports for the response.
- **Accept-Language: en-US,en;q=0.9**: States the client's preferred languages (US English, then English).